

BCSE303L

OPERATING SYSTEMS

Module 1 Lecture 2

Key concepts in Operating Systems

1. **Kernel**
2. **Process Scheduler**
3. **Memory Manager**
4. **File System**
5. **User Interface**
6. **Device drivers**

Key concepts in Operating Systems

The **kernel** is the core component of an operating system that manages the system's resources and provides low-level services to other components.

- It manages the operations of memory and CPU time.
- It is responsible for various tasks such as *disk management, task management, and memory management*.

The **process scheduler** is responsible for managing the allocation of CPU time to processes and ensuring that all processes get a fair share of the CPU.

The **memory manager** is responsible for managing the allocation of memory to processes and ensuring that processes do not interfere with each other's memory.

Key concepts in Operating Systems

The **file system manager** is responsible for organizing and managing the storage and retrieval of data on secondary storage devices such as hard disks.

- It is a system software responsible for the creation, deletion, modification of the files and managing their access, security and the resources used by them.
- These functions are performed in collaboration with the Device Manager.

Device drivers are software components that provide an interface between the operating system and specific hardware devices such as printers, network cards, and displays.

- It enable OS and other computer programs to access hardware functions without needing to know precise details about the hardware being used.

The **user interface** provides a way for the user to interact with the operating system and perform tasks such as launching programs and managing files.

System Calls

System calls provide an interface to the services made available by an operating system.

These calls are generally available as routines written in C and C++, although certain low-level tasks may have to be written using assembly-language instructions.

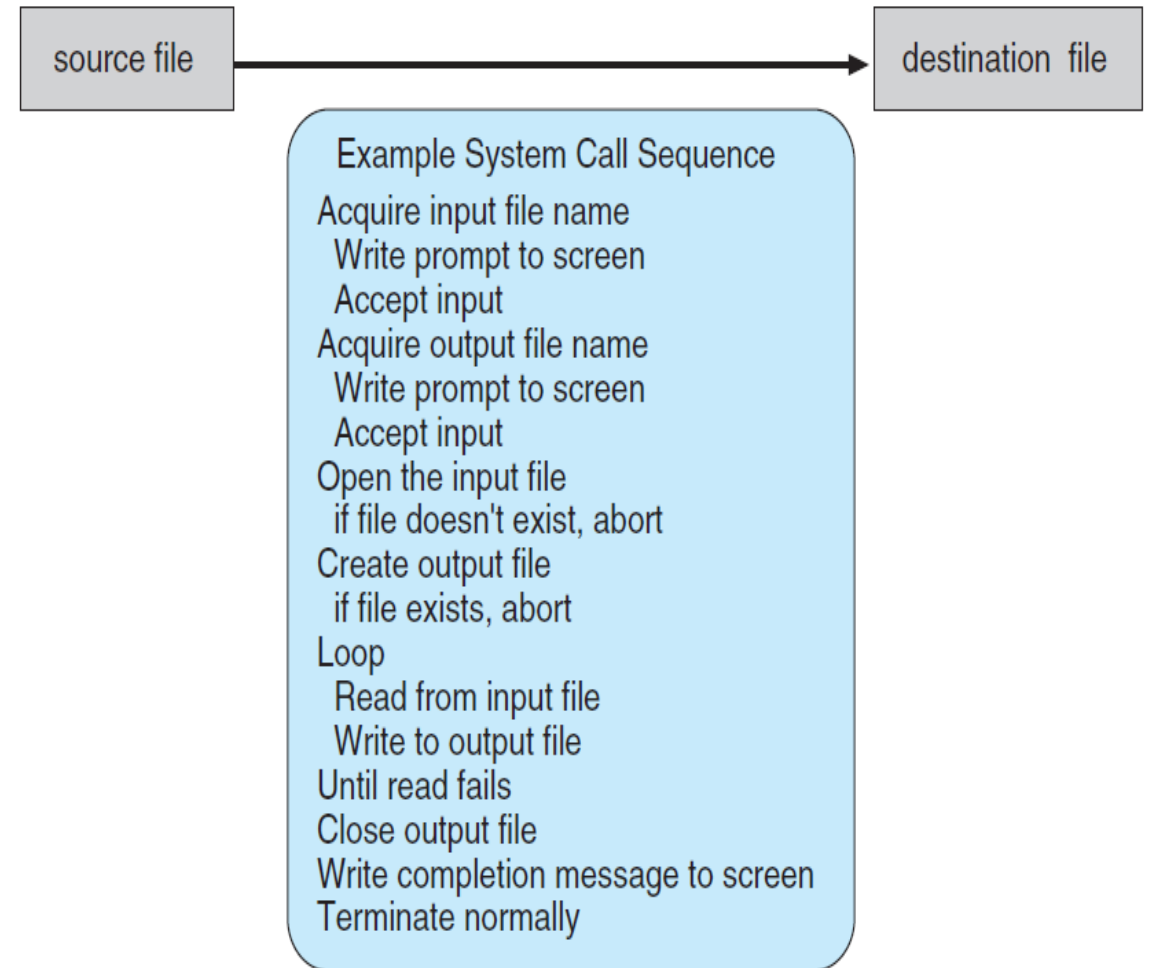


Figure: Example of how system calls are used

Structuring Operating system

1. Simple / Monolithic Structure

Place all of the functionality of the kernel into a single, static binary file that runs in a single address space.

Example – MS DOS and the original UNIX operating system,

- *In MS-DOS, the interfaces and levels of functionality are not well separated.*

UNIX OS has two separable parts: *the kernel and the system programs*. The kernel is further separated into a *series of interfaces and device drivers*.

The kernel provides the file system, CPU scheduling, memory management, and other operating- system functions through system calls.

The entire operating system is working in kernel space

Structuring Operating system

1. Simple / Monolithic Structure

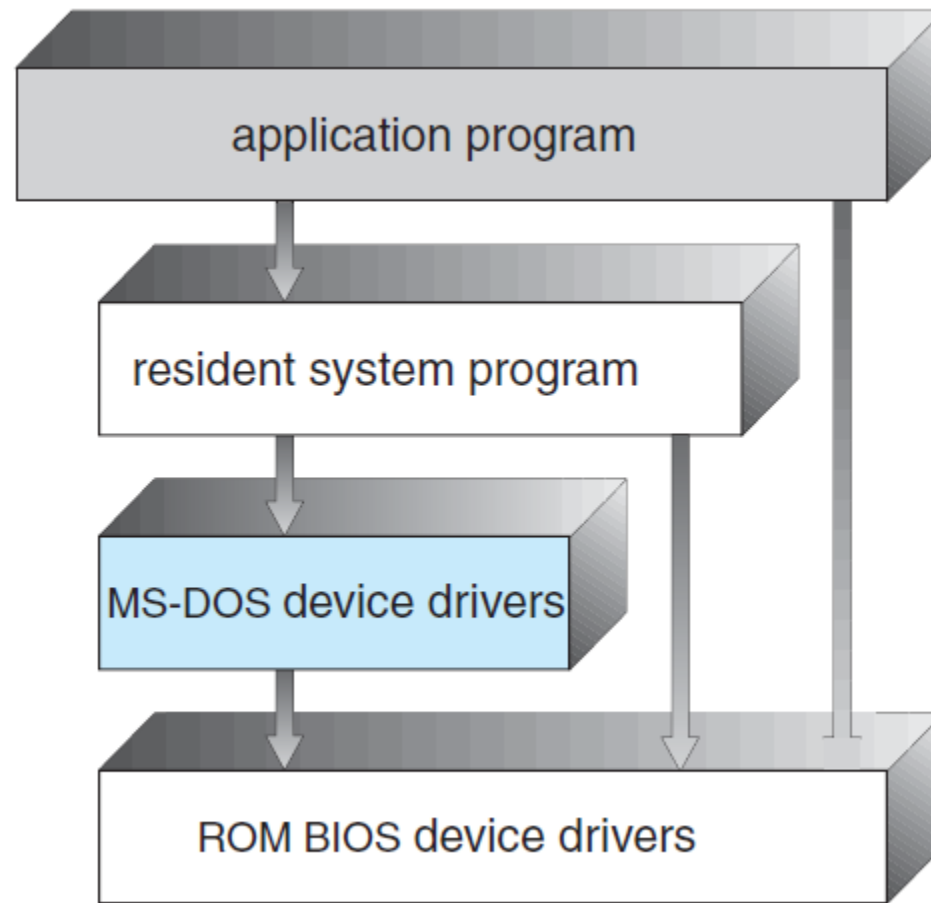


Figure: MS-DOS layer structure

Structuring Operating system

1. Simple / Monolithic Structure

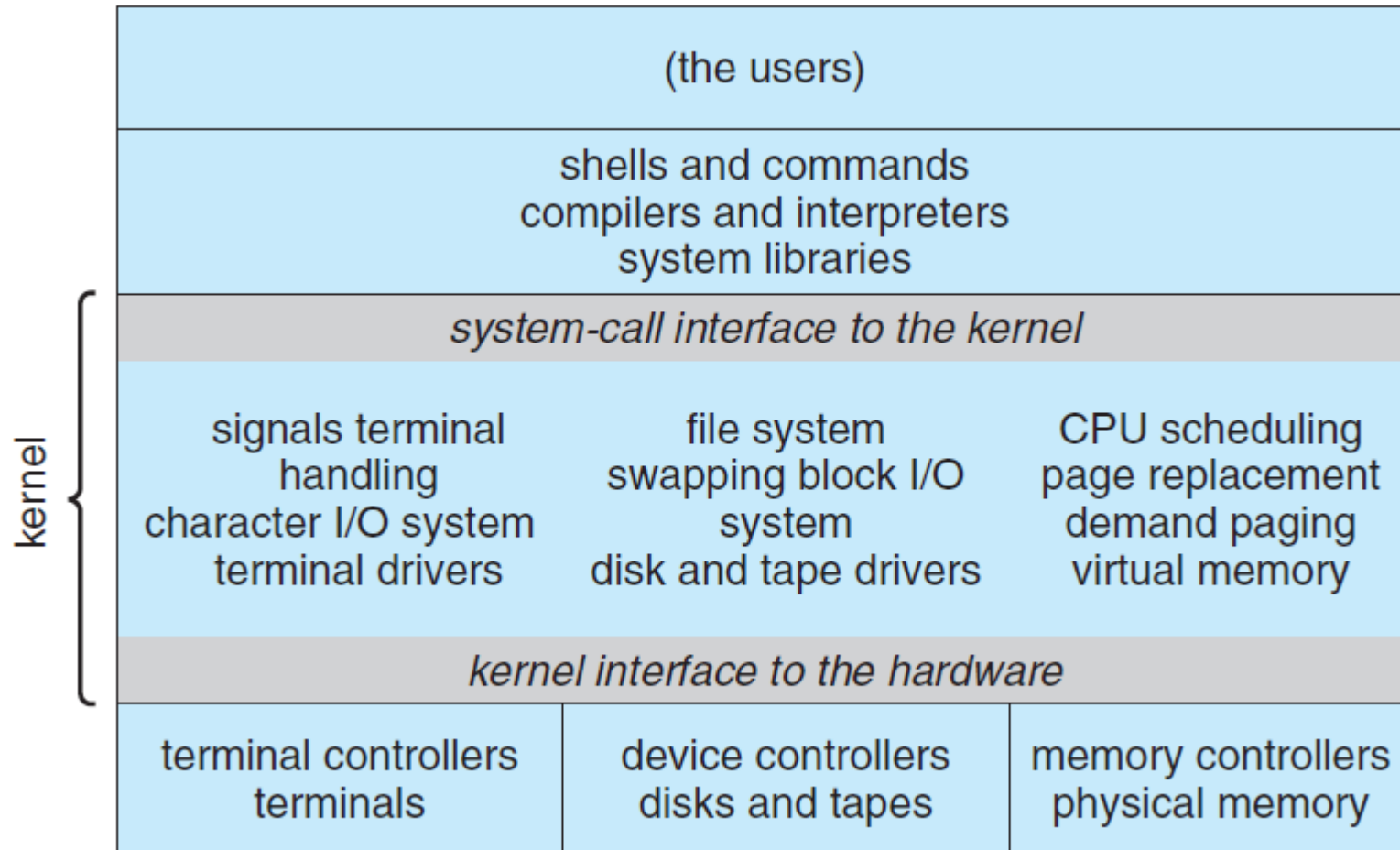


Figure:
Traditional UNIX
system structure

Structuring Operating system

1. Simple / Monolithic Structure

All code runs in kernel space, with no separation between kernel and user-level processes.

The kernel is designed to provide all operating system services, including memory management, process scheduling, device drivers, and file systems, in a single, large binary file.

Advantages:

- Simple to implement when functionalities are limited
- Provides high performance since system calls can be made directly to the kernel without the overhead

Structuring Operating system

1. Simple / Monolithic Structure

Dis-advantages:

- Less secure and stable
- Lack of hardware support
- Since all code runs in kernel space, any vulnerabilities or bugs in the kernel can potentially affect the entire system.
- If a user-level process crashes, it can bring down the entire system since there is no separation between kernel and user-level processes.

Structuring Operating system

2. Layered Structure

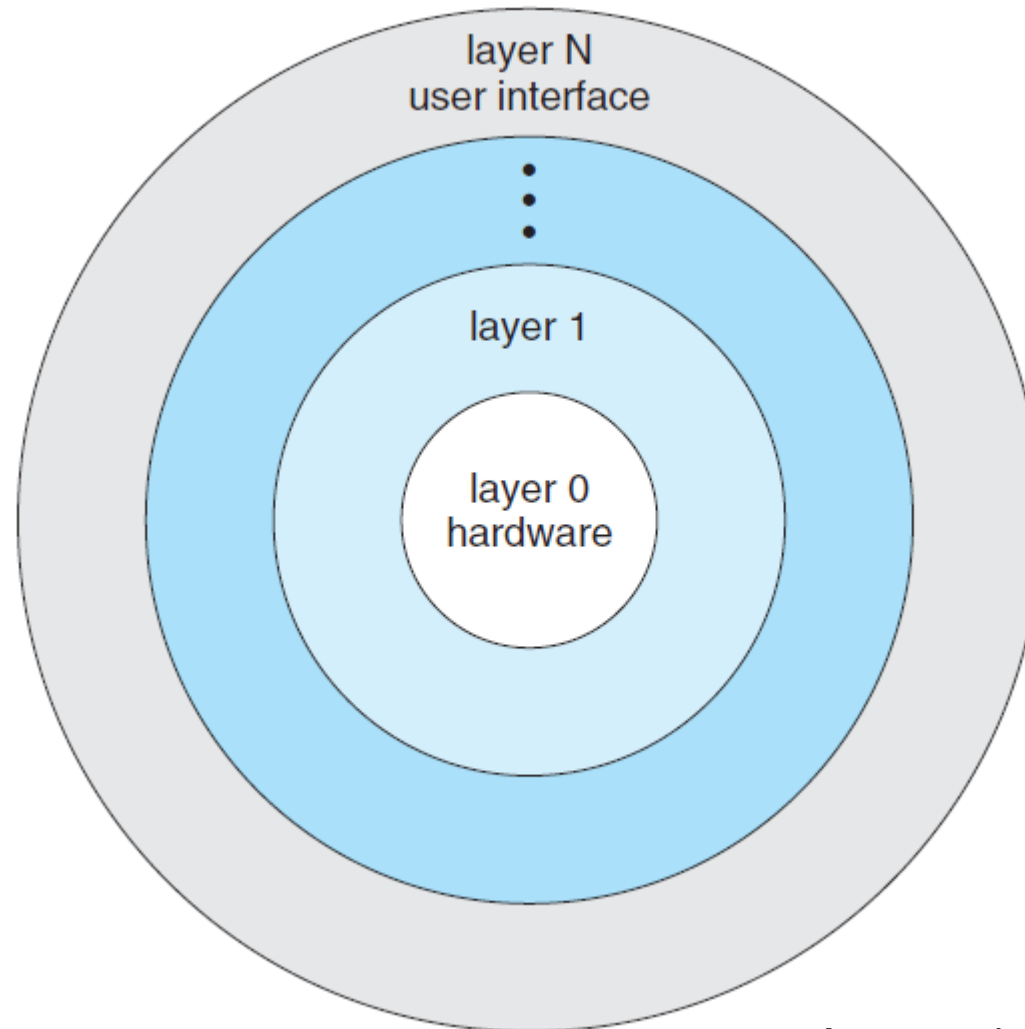


Figure: A layered operating system

Structuring Operating system

2. Layered Structure

In layered approach, the operating system was broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.

The different services of the operating system are split into various layers.

Each layer is responsible for performing a specific function and can only access the services provided by the layers below it.

For example: *layer M—consists of data structures and a set of routines that can be invoked by higher-level layers. Layer M, in turn, can invoke operations on lower-level layers.*

Structuring Operating system

2. Layered Structure

Each layer is implemented only with operations provided by lower-level layers.

A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

Hence, each layer *hides the existence of certain data structures, operations, and hardware from higher-level layers.*

Structuring Operating system

2. Layered Structure

Advantages:

- *Changes in one component affect only that component, and no others.*
- *Simplicity of construction and debugging, making it easier to develop, test, and maintain.*

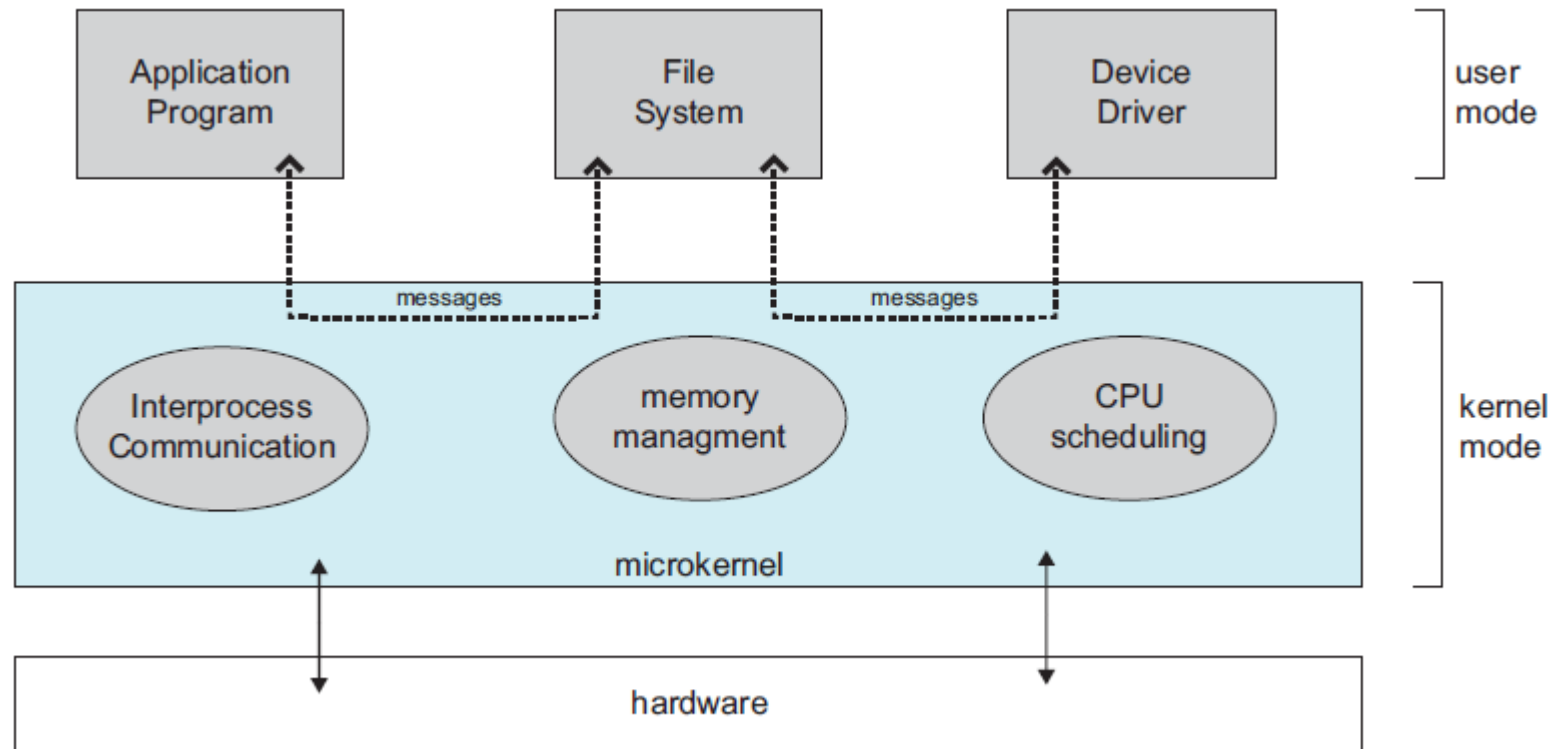
Dis-advantages:

- *The challenges of appropriately defining the functionality of each layer.*
- *The overall performance of such systems is poor due to the **overhead** of requiring a user program to traverse through multiple layers to obtain an operating-system service.*

Recommendation: *Fewer layers with more functionality are being designed*

Structuring Operating system

3. Microkernels Structure



Structuring Operating system

3. Microkernels Structure

In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called **Mach** that modularized the kernel using the microkernel approach.

This method removes all nonessential components from the kernel and implementing them as system and user-level programs. Thus the kernel is smaller now.

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space.

Communication is provided through **message passing**

Structuring Operating system

3. Microkernels Structure

Advantages

- It makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.
- The resulting operating system is easier to port from one hardware design to another.
- Provides more security and reliability, since most services are running as user

Dis-advantages

- The performance of microkernels can suffer due to increased system-function overhead
- Operating system may have to switch from one process to the next to exchange the messages. The overhead involved in copying messages and switching between processes has been the largest impediment to the growth of microkernel-based operating systems.

Structuring Operating system

4. Modules / Loadable kernel modules

The best current methodology for operating-system design

The kernel has a set of core components and can link in additional services via modules, either at boot time or during run time.

A **Loadable Kernel Module (LKM)** is an object file that contains code to extend the running kernel, or so-called base kernel, of an operating system.

Used to add support for new hardware (as device drivers), file systems, or system calls.

The idea of the design is for the kernel to provide core services, while other services are implemented dynamically, as the kernel is running.

Structuring Operating system

4. Modules / Loadable kernel modules

It uses object-oriented approach

When the functionality provided by an LKM is no longer required, it can be unloaded in order to free memory and other resources.

Advantages

- Allows for greater flexibility and modularity in the design of an operating system
- Functionality can be loaded and unloaded as and when required
- It doesn't require recompiling the kernel every time a change was made
- Modules do not need to invoke message passing in order to communicate

Example: Solaris, Linux, Mac OS X, Windows

Structuring Operating system

4. Modules / Loadable kernel modules

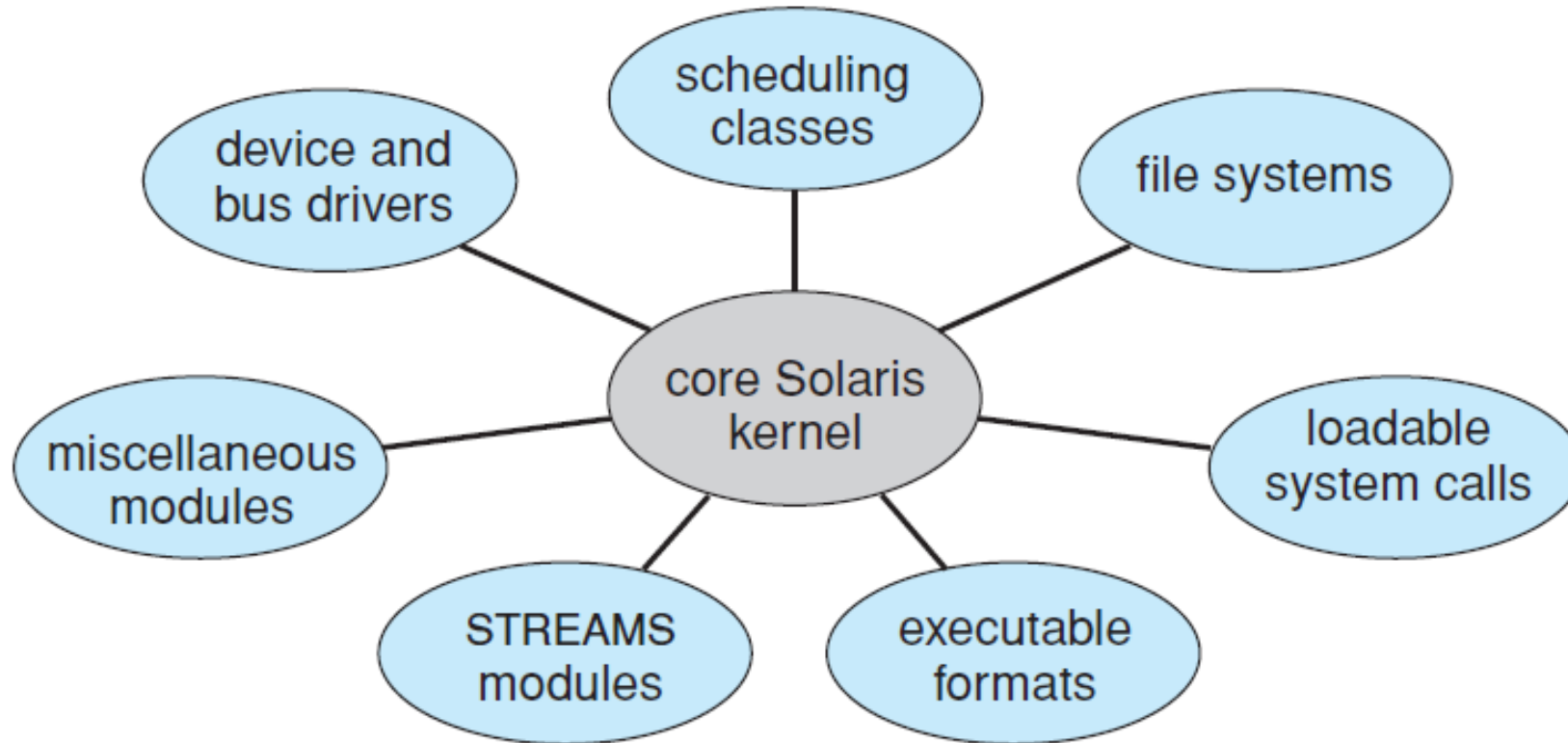


Figure: Solaris loadable modules.

Structuring Operating system

5. Hybrid Systems

*Most modern operating systems are actually **not one pure model***

- *Hybrid combines multiple approaches to address performance, security, usability needs*
- *Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality*
- *Windows mostly monolithic, plus microkernel for different subsystem personalities*

Structuring Operating system

5.1 macOS and iOS

Apple's macOS operating system is designed to run primarily on desktop and laptop computer systems, whereas iOS is a mobile operating system designed for the iPhone smartphone and iPad tablet computer.

The general architecture of these two systems has various layers include the following:

- 1. **User experience layer** – Aqua vs Springboard*
- 2. **Application framework layer** – Cocoa vs Cocoa touch*
- 3. **Core frameworks** - support graphics and media including, Quicktime and OpenGL.*
- 4. **Kernel** – Darwin (Mach microkernel and BSD UNIX kernel)*

Structuring Operating system

5.1 macOS and iOS

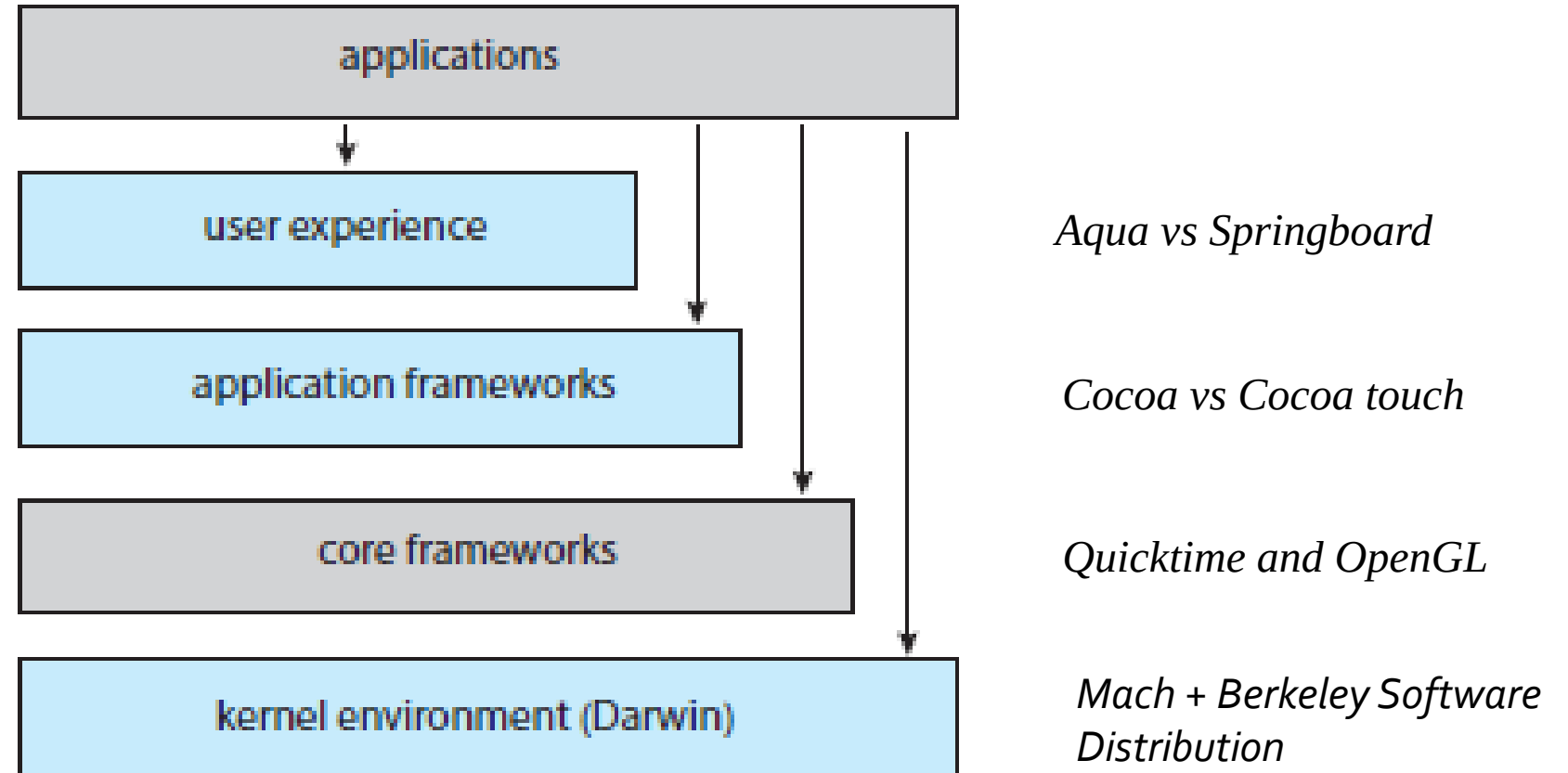


Figure: Architecture of Apple's macOS and iOS operating systems

Structuring Operating system

5.1 macOS and iOS

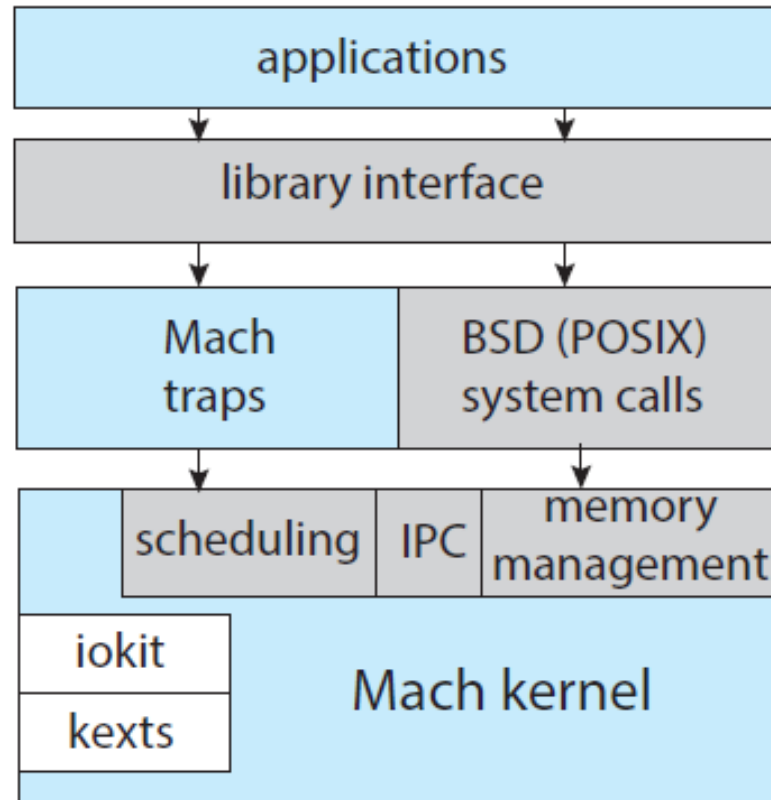


Figure: The structure of Darwin

Structuring Operating system

5.1 macOS and iOS

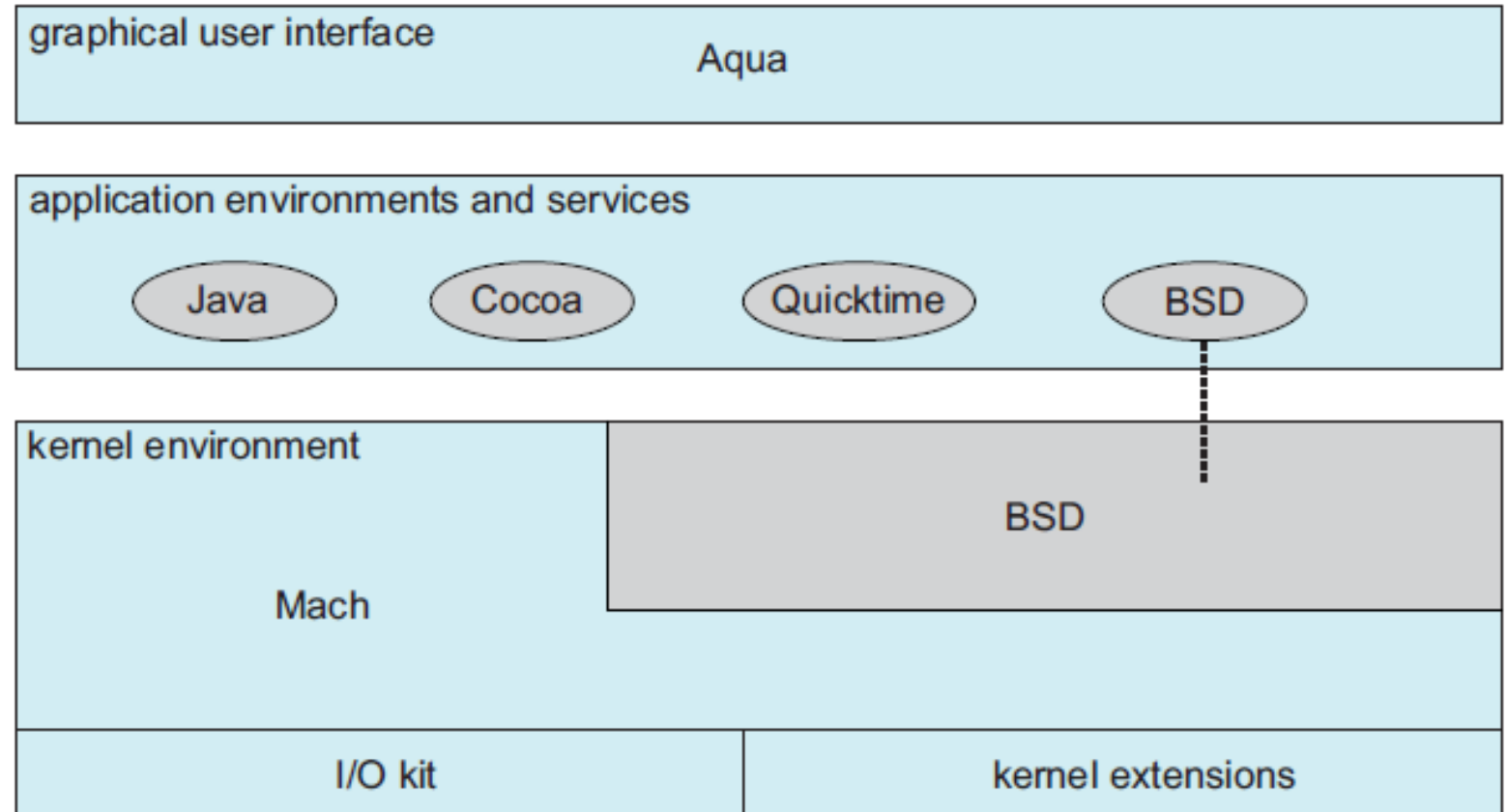


Figure: The Mac OS X structure

Structuring Operating system

5.1 macOS and iOS



Figure: Architecture of Apple's iOS

Structuring Operating system

5.1 macOS and iOS

User experience layer defines the software interface that allows users to interact with the computing devices.

Application frameworks layer includes the Cocoa and Cocoa Touch frameworks, which provide an API for the Objective-C and Swift programming languages.

Core frameworks layer defines frameworks that support graphics and media including, Quicktime and OpenGL.

Kernel environment also known as Darwin which includes the Mach microkernel and the BSD UNIX kernel.

Structuring Operating system

5.1 macOS and iOS

Some significant distinctions between macOS and iOS include the following:

*Because macOS is intended for desktop and laptop computer systems, it is compiled to run on **Intel architectures**. iOS is designed for mobile devices and thus is compiled for **ARM-based architectures**.*

*The iOS operating system is generally much **more restricted** to developers than macOS. For example, iOS **restricts access to POSIX and BSD APIs** on iOS, whereas they are openly available to developers on macOS.*

Structuring Operating system

5.1 macOS and iOS

Darwin, which uses a hybrid structure. It is a layered system that consists of the Mach *microkernel and the BSD UNIX kernel*.

Darwin provides two system-call interfaces: Mach system calls and BSD system calls.

These system calls has a rich set of libraries to provide networking, security, and programming language support.

Mach provides fundamental operating system services, including memory management, CPU scheduling, and interprocess communication

Summary

Structure	Advantage	Disadvantage	Example
Monolithic	High performance	Difficult to maintain and debug	Linux, early Unix
Microkernel	Better modularity, security	Potential performance overhead	QNX, MINIX, L4
Layered	Clear separation of concerns	Performance overhead, strict layer boundaries	THE, early Unix
Loadable Modules Kernel	Dynamic loading/unloading, flexibility	Security risks, complexity	Linux, Windows
Hybrid	Performance, modularity	Complexity, potential trade-offs	Windows NT, macOS (XNU)

References

1. **Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.**